

THE APPLICATION IS THE BUILD

GTM Engineer

Case Study: The 80-Blogs-a-Week Engine

THE BRIEF IN ONE LINE

Design and partially build the engineering system that lets NeevCloud publish **80 blog posts per week** — indexed, ranking, and revenue-attributable — without the quality, crawl, or brand penalties that kill most programmatic content operations.

TIME LIMIT	7 calendar days from receipt
DELIVERABLES	1) A detailed technical plan (document) 2) A presentation that defends the plan (deck)
ROLE LENS	This is an engineering role, not a marketing role. We are hiring the person who turns a GTM process into code that runs at scale.
PASS RATE	Designed to be hard. Most submissions will not pass. Read every section before you start building.

o. How to Read This Document

This case study is deliberately over-specified in some places and deliberately silent in others. Both are signals. Where we give you exact numbers, we expect you to use them. Where we are silent, we expect you to state your assumptions, defend them, and move. Waiting for perfect specs before building is itself a failing answer.

Read the whole thing first. There is a trap embedded in the headline number. If your plan treats '80 blogs per week' purely as a content-generation throughput problem, you have already failed the core test of this role. We will tell you why later — but we want to see if you catch it before we do.

1. Business Context

NeevCloud is an AI-first cloud and data infrastructure company built in India for enterprises, government, and high-growth startups. We sell sovereign cloud, GPU compute for AI workloads, data-centre capacity, and enterprise infrastructure programs. Our buyers are technical and senior: platform engineering leads, CTOs, heads of infrastructure, AI/ML leads, and procurement in regulated sectors (BFSI, public sector, healthcare).

Organic search is strategically important to us for three reasons:

1. Our buyers research extensively before they ever talk to sales. They search for comparisons, benchmarks, compliance explainers, and migration guides. We are currently invisible for most of these queries.
2. Paid acquisition in cloud/GPU is brutally expensive and competitive against hyperscalers with effectively infinite budgets. We cannot win on CPC. We can win on depth, specificity, and sovereign/India-specific intent.
3. Content compounds. A ranking page is an asset that generates pipeline for years. This is the single highest-leverage organic motion available to us — if, and only if, it is engineered correctly.

Leadership has set a target: ramp organic content publishing to a sustained 80 posts per week. Your job is to engineer the system that makes that volume produce pipeline rather than penalties. You are not the writer. You are the person who builds the machine.

WHAT '80 PER WEEK' ACTUALLY MEANS

80/week = ~347/month = ~4,160 indexable URLs per year. At that rate you are operating at a scale where Google's site-level quality systems, crawl economics, and internal-link topology dominate outcomes. Throughput is the easy part. Surviving it is the hard part.

2. The Problem You Must Solve

Build the engineered, end-to-end system that takes NeevCloud from its current state (sporadic, manual, low-volume publishing) to a sustained 80 high-quality, indexable, ranking-capable posts per week. ‘Engineered’ is the operative word: we want pipelines, APIs, webhooks, data models, and measurement — not a description of a content calendar and a writer rota.

2.1 Current State (your starting assumptions)

Dimension	Where we are today
CMS	Headless CMS available (assume one with a REST + GraphQL API and webhooks; name the one you would actually use).
Current output	~3–5 posts/week, written manually, published by hand, inconsistent structure.
Domain authority	Low-to-moderate. The domain is not a trusted authority in cloud/AI yet. Treat trust as scarce.
Indexing	No log-file analysis, no crawl-budget management, sitemaps are stale and monolithic.
Tracking	GA4 installed but no content-to-pipeline attribution. GSC connected but unused beyond eyeballing.
Stack access	You may assume access to Clay, Apollo/Instantly/Smartlead, n8n/Make/Zapier, HubSpot or Salesforce, Google & Meta Ads APIs, GA4/GSC/Looker, and any LLM APIs you choose.

2.2 Hard Constraints

These are non-negotiable. A plan that ignores any of them fails.

- **Quality floor:** every published URL must be defensibly useful to a senior technical buyer. ‘Spun’ or thin AI content is an automatic fail. You must build quality control into the pipeline, not bolt it on.
- **Indexation, not just publication:** a post that publishes but never gets crawled or indexed does not count toward the 80. Your system is measured on indexed, eligible-to-rank URLs.
- **No manual bottleneck:** the system must not require a human to touch each of the 80 posts individually. Humans review, approve, and steer — they do not hand-assemble.
- **Attribution:** you must be able to tie organic content to pipeline (MQL → SQL → opportunity), not just sessions. Wire the content stack to the CRM.
- **Brand & compliance safety:** we sell to government and BFSI. A hallucinated benchmark, a leaked internal figure, or a non-compliant claim is a business risk. Build guardrails.
- **Budget realism:** assume a finite monthly budget for tooling + LLM tokens + freelance editors. State your number and your unit economics (cost per published, indexed post).

3. The SEO Deep Dive (read this carefully)

This is the heart of the case study. We are testing whether you understand SEO as an engineering and systems problem, not a list of ‘best practices.’ Your plan must demonstrate working knowledge of every area below. We have phrased each as the question we will be silently asking while we grade you.

3.1 The volume trap (the thing most candidates miss)

THE CORE TEST

Publishing 80 posts/week is trivial. Getting 80 posts/week indexed, ranking, and not dragging down the rest of the domain is extraordinarily hard. Google evaluates quality at the site level, not just the page level. Flood a low-trust domain with 4,000+ pages a year and you risk site-wide demotion under the Helpful Content / core quality systems, partial indexation, and crawl-budget starvation of your money pages.

If your answer does not confront this directly, it does not pass. A strong answer may even argue that 80/week of editorial long-form is the wrong shape, and that the volume should come from a programmatic SEO substrate (templated, data-backed pages over a real dataset) plus a smaller spine of editorial pillars. Make the argument with numbers.

3.2 Crawl budget & indexation economics

- How will you ensure Googlebot actually crawls 80 new URLs/week on a low-authority domain? What is crawl budget, what determines it, and how do you engineer for it?
- How do you use log-file analysis to see what Googlebot is actually doing (crawl frequency, crawl waste on parameters/redirects/404s, crawl depth)? What pipeline pulls and parses these logs?
- Sitemap architecture at scale: why monolithic sitemaps fail, how you shard them, how you use <lastmod> honestly, and how you read the indexed-vs-submitted delta in GSC as a health metric.
- The Indexing API and sitemap pings: where they legitimately apply, where they do not, and why ‘just submit every URL to the Indexing API’ is a wrong answer for blog content.
- How do you detect and fix index bloat, partial indexation, and ‘Discovered – currently not indexed’ / ‘Crawled – currently not indexed’ states programmatically?

3.3 Site-level quality, E-E-A-T & the Helpful Content problem

- Explain, at a mechanism level, how you keep average per-URL quality high enough that 80/week does not trip site-wide quality demotion. What is your automated quality gate, and what does it actually measure?
- E-E-A-T for an infrastructure brand: how do you engineer experience, expertise, authoritativeness and trust signals into a templated pipeline (author entities, citations, original data, schema, first-party benchmarks)?
- How do you prevent thin, near-duplicate, or cannibalising pages when generating at volume? Define your near-duplicate detection and keyword-cannibalisation detection as automated checks.

- What is your prune/refresh/consolidate strategy? At 4,000+ pages/year, content decay and pruning underperformers is part of the system, not an afterthought.

3.4 Information architecture, internal linking & topical authority

- Topic clusters / pillar-and-spoke: how do you build topical authority deliberately rather than publishing 80 disconnected posts? Model this as a graph.
- Internal linking at scale is an engineering problem. How does a new post automatically earn relevant internal links from and to existing content? Describe the algorithm (embeddings? entity matching? a link graph with relevance scoring?). How do you avoid orphan pages and control link equity flow?
- URL taxonomy, canonicalisation strategy, and how you prevent the programmatic layer from generating crawlable infinite spaces (faceted/parameter traps).

3.5 Technical SEO & rendering

- Rendering: SSR vs SSG vs CSR for content at this volume — what do you choose and why, given crawl and indexation goals? What breaks if it is client-rendered?
- Core Web Vitals (LCP, INP, CLS) as a build-time gate: how do you stop a slow or unstable template from shipping?
- Structured data at scale: which schema.org types (Article, FAQPage, BreadcrumbList, Organization, etc.), how you generate and validate it programmatically, and how you monitor rich-result eligibility in GSC.
- Hreflang / India-and-global targeting if relevant; XML vs HTML sitemaps; robots directives; the difference between noindex, canonical, and robots.txt disallow, and when each is the right tool.

3.6 Keyword → brief → generation → QA pipeline

- Where does the topic queue come from? Describe the data pipeline: keyword/intent sourcing (GSC query mining, SERP gap analysis, third-party APIs), clustering, prioritisation by opportunity (volume × intent × difficulty × business value), and de-duplication against what you have already published.
- How does a queued topic become a structured brief, then a draft, then a QA-passed publishable asset? Name the steps, the models, the prompts-as-code, the validation gates, and the human-in-the-loop checkpoints.
- How do you stop the LLM from hallucinating numbers, inventing sources, or leaking confidential NeevCloud data into a public post? This is a hard requirement, not a nice-to-have.

3.7 Measurement, attribution & the kill-switch

- Define the metrics that matter and reject vanity metrics. We expect: indexed-URL rate, time-to-index, ranking-keyword growth, impressions → clicks → assisted pipeline, and cost per indexed-ranking-page. Track rate of change, not just absolute counts.
- How is the content layer wired to GA4, GSC, and the CRM (HubSpot/Salesforce) so you can attribute organic content to pipeline? Describe the data flow and the join keys.

- What is your automated kill-switch / circuit-breaker? If site-wide quality signals start degrading, how does the system detect it and throttle or roll back before it damages the domain?

4. Engineering Expectations

This is where tool-configurators and engineers diverge. We want to see systems thinking expressed in architecture, not a description of clicking around in a SaaS UI.

4.1 What we expect to see

- **An architecture diagram** of the whole system: sources → queue → generation → QA gates → CMS → indexation → measurement → feedback loop. Show the data model and the integration points (APIs, webhooks, queues).
- **Real integration design:** which calls hit which APIs, what the webhook payloads carry, how state is stored, how failures are retried, where idempotency lives. n8n/Make/Zapier may orchestrate, but show where you would drop to code (Python/JS) and why.
- **At least one component built for real.** Pick the highest-risk or highest-leverage piece (e.g. the QA/quality-gate service, the internal-linking engine, the indexation monitor, or the keyword-clustering pipeline) and actually build a working prototype. Share the code (repo or gist). We would rather see one thing that runs than ten things described.
- **Unit economics:** cost per published-and-indexed post, throughput math, and where the system breaks if you push past 80/week.
- **Failure modes & guardrails:** what goes wrong, how you detect it, how the system degrades safely.

4.2 What will sink your submission

- **Treating it as a pure throughput problem** ('wire GPT to n8n to the CMS and loop 80 times') with no quality, crawl, or trust strategy.
- **Vague hand-waving** ('we'll use AI to ensure quality') with no defined mechanism, metric, or gate.
- **No-code-only answers** that never touch an API, a data model, or a line of code.
- **Vanity metrics** ('we'll track traffic and engagement') with no path to pipeline.
- **Ignoring the volume trap** in Section 3.1.

5. Your Deliverables

Two artefacts. Both are graded. Do not pad either one.

Deliverable 1 — The Detailed Technical Plan (document)

A written plan that an engineering and GTM leader could read and immediately understand how the system works and why it will not break the domain. We are not prescribing length — we are prescribing density. Every section earns its place. At minimum, cover:

4. Executive summary, including your explicit position on the Section 3.1 volume trap and the content shape you recommend.
5. Target-state architecture: full system diagram, data model, integration map (APIs/webhooks/queues), and tech choices with justifications.
6. The SEO engineering strategy: crawl/indexation, site-level quality & E-E-A-T, internal-linking engine, technical SEO/rendering, structured data — addressing every sub-area in Section 3.
7. The content pipeline: topic sourcing → prioritisation → brief → generation → multi-stage QA → publish → index → measure, with the human-in-the-loop checkpoints marked.
8. Quality, safety & compliance guardrails (hallucination control, confidential-data leakage prevention, claim verification).
9. Measurement & attribution model: metric tree, content-to-pipeline wiring, dashboards, and the automated kill-switch.
10. Unit economics and a capacity model (how throughput, cost, and quality trade off).
11. A 30/60/90-style rollout: what you ship first, how you ramp to 80/week safely, and the leading indicators you watch at each stage.
12. The prototype: link to the one component you actually built, with a short note on what it does and how to run it.
13. Risks, failure modes, and what you deliberately chose NOT to do (and why).

Deliverable 2 — The Presentation (deck)

A presentation that defends the plan to a skeptical room (founders + engineering + GTM leadership). It must stand on its own and survive hard questions. We suggest roughly 10–15 slides. Make it cover:

- The problem framed correctly — including the volume trap and why naive 80/week is dangerous.
- Your recommended content shape and the reasoning (programmatic substrate vs editorial spine vs hybrid).
- The system architecture, on one clear diagram you can talk to.
- The SEO engineering decisions that de-risk volume: crawl/indexation, site-level quality, internal linking, technical SEO.
- The content pipeline and where humans intervene.
- Measurement, attribution, and the kill-switch.
- Unit economics and the ramp plan to 80/week.

- Your prototype — show it working.
- Risks and trade-offs, stated honestly.

A NOTE ON THE DECK

We are not grading visual polish. We are grading whether you can compress a complex system into a defensible argument and answer ‘why’ at every layer. A clear deck from someone who understands the system beats a beautiful deck that falls apart on the second question.

6. How We Will Grade You

Total 100 points. The bar is high by design. We expect most submissions to fall short on Sections A and B — that is the whole point of this exercise.

Area	Weight	What earns the points
A. SEO depth & the volume trap	30	Confronts site-level quality, crawl/indexation economics, and the right content shape. This is the make-or-break section.
B. Engineering rigour	25	Real architecture, data model, integration design, and a working prototype — not no-code hand-waving.
C. Quality, safety & guardrails	12	Mechanisms for quality control, hallucination/confidentiality control, and the kill-switch.
D. Measurement & attribution	13	Metric tree that rejects vanity, content-to-pipeline wiring, rate-of-change thinking.
E. Unit economics & ramp realism	10	Honest cost model, capacity math, and a safe ramp to 80/week.
F. Communication & defensibility	10	Plan and deck are clear, dense, and survive hard questions.

BONUS SIGNAL (UNCAPPED, CAN RESCUE A BORDERLINE SUBMISSION)

Tell us about a real GTM system you have shipped end-to-end — an outreach machine, a content pipeline, an ad-to-CRM integration. What broke, what you measured, what you would build differently. As the JD says: that is the application.

7. Rules of Engagement

- **Use any tools you like.** AI assistants included — we assume you use them well. We are testing judgement, not memorisation.
- **State your assumptions.** Where we were silent, fill the gap explicitly and defend the choice. Do not email us for clarification on things you can reasonably assume.
- **Realism over completeness.** A focused, defensible system beats an exhaustive wish-list. Tell us what you would cut.
- **Show working code for at least one component.** A repo or gist link inside the plan.
- **Submit both files by the 7-day deadline.** Late or single-file submissions are scored as incomplete.

The channel changes. The expectation doesn't: ship fast, measure everything, iterate. Show us the machine you would build.